

Capital Group Global Dividend Watch Tool

A JavaScript widget for analysing and visualising dividend payment data from CSV files. It provides interactive filtering, side-by-side comparison, and multiple visualisation options tailored to financial datasets.

Table of contents

- 1. [Quick start](#)
- 2. [Features](#)
- 3. [Installation](#)
- 4. [Usage](#)
- 5. [Technical architecture](#)
- 6. [API reference](#)
- 7. [Customisation](#)
- 8. [Troubleshooting](#)
- 9. [Performance considerations](#)
- 10. [Support](#)
- 11. [Version history](#)
- 12. [Licence](#)

Quick start

```
<!DOCTYPE html>
<html>
<head>
  <title>Capital Group Global Dividend Watch</title>
</head>
<body>
  <h1>Dividend Dashboard</h1>
  <script src="js/dividend-tool.js" data-csv="data/dividend-data.csv"></script>
</body>
</html>
```

CSV requirements

Your CSV file should contain the following columns:

- **"Dividend, USD"**: Numeric dividend amount in USD (column name can contain "dividend")
- **QUARTER**: Quarter identifier (format: "YYQ#" like "23Q1", "10Q3")
- **REGION**: Geographic region name (e.g., "Canada", "Europe ex-UK", "US")
- **COUNTRY**: Country name (e.g., "Canada", "Germany", "United States")
- **SECTOR**: Industry sector name (e.g., "Consumer Cyclical", "Financial", "Technology")
- **SUBSECTOR**: Industry subsector name (e.g., "Auto", "Banks", "Software & IT Services")
- **YEAR**: Full year (e.g., "2010", "2023")

Example CSV

Features

Data analysis

- CSV parsing and validation
- Multi-dimensional filtering (region, country, sector, subsector)
- Time-based analysis (quarters, years, custom ranges)
- Dataset comparison (A vs B)
- Real-time calculations as filters change

Visualisation

- Interactive charts (Chart.js)
- Responsive data tables
- Quarterly or yearly views
- Number formatting in billions, millions, or full values
- Progressive disclosure (hidden until "Show Data" clicked)
- Collapsible chart data tables

- Works on desktop and mobile

User interface

- Year and quarter dropdowns for date range selection
- Cascading filters (dependent selects)
- Smart dropdown visibility (hides single-option fields)
- Info tooltips (Popper.js)
- Shadow DOM isolation to prevent CSS clashes
- Customizable display labels for sectors, regions, and subsectors

Example CSV

```
REGION,COUNTRY,SECTOR,SUBSECTOR,YEAR,QUARTER,"Dividend, USD"
Canada,Canada,Consumer Cyclical,Auto,2010,10Q3,84533630.4
Europe ex-UK,Germany,Technology,Software & IT Services,2010,10Q2,2300000000
US,United States,Financial,Banks,2023,23Q4,850000000
```

Usage

Basic implementation

```
<!DOCTYPE html>
<html>
<head>
  <title>Dividend Analysis</title>
</head>
<body>
  <h1>Dividend Analysis Dashboard</h1>

  <!-- The tool will be inserted after this script tag -->
  <script src="js/dividend-tool.js" data-csv="data/2507-Data-Extract.csv"></script>

</body>
</html>
```

Display modes

The tool displays data across multiple quarters/years with:

- Interactive date range dropdowns (year and quarter selectors)
- Choice between quarterly or yearly aggregation
- Visual charts and detailed tables
- Progressive disclosure - data visualization hidden until "Show Data" button clicked

Filtering options

Geographic Filtering

- **Region:** High-level geographic areas (US, Europe, Asia-Pacific, etc.)
- **Country:** Specific countries within regions (cascading filter)

Industry Filtering

- **Sector:** Major industry categories (Financial, Technology, Healthcare, etc.)
- **Subsector:** Detailed industry classifications (cascading filter)

Time Filtering

- **Date Range:** Use year and quarter dropdowns to select custom ranges
- Quarter dropdowns automatically hide when viewing by years

Comparison mode

1. **Enable Compare Mode:** Click the "+ add dataset" link below Dataset A
2. **Configure Dataset A:** Use the first row of filters
3. **Configure Dataset B:** Use the second row of filters that appears

4. **View Results:** Charts and tables show both datasets side-by-side
5. **Remove Dataset B:** Click the "- remove dataset" link

Number formatting

Choose from three display formats:

- **USD Billions:** \$2.50bn (recommended for large datasets)
- **USD Millions:** \$2,500.00m
- **Full USD:** \$2,500,000,000.00

CSV requirements

- "Dividend, USD" — numeric dividend amount
- QUARTER — format YYQ# (e.g. 23Q1)
- REGION — e.g. *Europe ex-UK*
- COUNTRY — e.g. *Germany*
- SECTOR — e.g. *Technology*
- SUBSECTOR — e.g. *Software & IT Services*
- YEAR — e.g. 2023

Example CSV

```
REGION,COUNTRY,SECTOR,SUBSECTOR,YEAR,QUARTER,"Dividend, USD"
Canada,Canada,Consumer Cyclical,Auto,2010,10Q3,84533630.4
Europe ex-UK,Germany,Technology,Software & IT Services,2010,10Q2,2300000000
US,United States,Financial,Banks,2023,23Q4,850000000
```

Usage

Basic implementation

See [Quick start](#).

Display modes

The tool displays data across multiple quarters/years with interactive date range dropdowns and choice between quarterly or yearly aggregation.

Filtering options

- **Geography:** Region → Country (cascading, auto-hides if only one option)
- **Industry:** Sector → Subsector (cascading, auto-hides if only one option)
- **Time:** Custom date range via year/quarter dropdowns

Comparison mode

1. Enable via "+ add dataset" link
2. Configure **Dataset A** filters
3. Configure **Dataset B** filters
4. View results side-by-side
5. Remove via "- remove dataset" link

Comparison mode

1. Enable **Compare mode** via checkbox
2. Configure **Dataset A** filters
3. Configure **Dataset B** filters
4. View results side-by-side

Number formatting

- \$2.50bn (billions)
- \$2,500.00m (millions)
- \$2,500,000,000.00 (full)

Technical architecture

Integration compatibility

- Pure ECMAScript, no frameworks required
- Shadow DOM isolation
- Self-contained (CDN dependencies)
- Works with CMSs (WordPress, Drupal) or standalone HTML
- Can be wrapped in React/Vue components

Dependencies

- **Papa Parse** (5.4.1): CSV parsing
- **Popper.js** (2.11.8): Tooltip positioning
- **Chart.js** (4.4.1): Data visualisation
- **External CSS**: Styles loaded from `css/dividend-tool.css`

Integration compatibility

This tool is designed for seamless integration with existing web ecosystems:

Framework Agnostic Design

- **Pure ECMA JavaScript**: No framework dependencies (React, Vue, Angular, etc.)
- **Shadow DOM Isolation**: Prevents CSS conflicts with host applications
- **Self-contained**: All dependencies loaded via CDN, no build process required
- **Universal Compatibility**: Works with any modern web platform or CMS

Integration Options

- **Direct HTML**: Simple script tag integration (current implementation)
- **React/Vue Components**: Can be wrapped in framework components
- **CMS Integration**: Compatible with WordPress, Drupal, custom CMSs
- **API Integration**: Can be enhanced to consume REST/GraphQL APIs instead of CSV

Production Considerations

- **Performance**: Optimized for datasets up to 50,000 rows
- **Security**: No server-side dependencies, client-side only
- **Scalability**: Can be enhanced with data pagination/virtualization
- **Maintenance**: Single JavaScript file, easy to update/modify

Shadow DOM implementation

- Uses Shadow DOM for style isolation
- Prevents CSS conflicts with host page
- Self-contained component architecture

Data processing

1. **CSV Parsing**: Loads and validates CSV data
2. **Normalization**: Standardizes column names and formats
3. **Quarter Processing**: Handles various date formats
4. **Filtering**: Real-time data filtering based on user selections
5. **Aggregation**: Sums dividend amounts across filtered results

Browser compatibility

- **Modern Browsers**: Chrome 53+, Firefox 63+, Safari 10.1+, Edge 79+
- **Mobile Support**: iOS Safari, Chrome Mobile, Samsung Internet
- **Features Used**: Shadow DOM, ES6 features, Flexbox, CSS Grid

Enterprise integration

Presentation Layer Integration

The tool is specifically designed to work with presentation layer frameworks:

```
// React Component Example
function DividendDashboard({ csvUrl }) {
  useEffect(() => {
    const script = document.createElement('script');
    script.src = '/js/dividend-tool.js';
    script.setAttribute('data-csv', csvUrl);
    document.body.appendChild(script);

    return () => document.body.removeChild(script);
  }, [csvUrl]);

  return <div id="dividend-tool-container"></div>;
}
```

API Enhancement Options

While currently CSV-based, the tool can be modified for API integration:

```
// Potential API integration modification
async function loadDataFromAPI(endpoint) {
  const response = await fetch(endpoint);
  const data = await response.json();
  // Transform API data to expected format
  return transformToExpectedFormat(data);
}
```

Deployment Flexibility

- **Static Hosting:** Can be deployed to CDNs, static sites
- **Server Integration:** Works with any backend technology
- **Microservice Architecture:** Can be enhanced as a standalone service
- **Content Delivery:** Optimized for fast loading and caching

Data processing

1. CSV parsing and validation
2. Normalisation of field names
3. Quarter/year handling
4. Real-time filtering
5. Aggregation across results

Browser compatibility

- Chrome 53+
- Firefox 63+
- Safari 10.1+
- Edge 79+
- Mobile Safari, Chrome Mobile, Samsung Internet

Enterprise integration

React component example

```
// React Component Example
function DividendDashboard({ csvUrl }) {
  useEffect(() => {
    const script = document.createElement('script');
    script.src = '/js/dividend-tool.js';
    script.setAttribute('data-csv', csvUrl);
    document.body.appendChild(script);

    return () => document.body.removeChild(script);
  }, [csvUrl]);

  return <div id="dividend-tool-container"></div>;
}
```

API integration enhancement

```
// Potential API integration modification
async function loadDataFromAPI(endpoint) {
  const response = await fetch(endpoint);
  const data = await response.json();
  // Transform API data to expected format
  return transformToExpectedFormat(data);
}
```

Deployment flexibility

- **Static hosting** – deploy to CDNs or static sites
- **Server integration** – works with any backend stack
- **Microservice architecture** – can be enhanced as a standalone service
- **Content delivery** – optimised for fast loading and caching

API reference

Configuration attributes

- **data-csv (required)**: path to CSV file

```
<script src="dividend-tool.js" data-csv="path/to/data.csv"></script>
```

CSS classes (inside Shadow DOM)

- `.bar-top` – toolbar with toggles
- `.bar-bottom` – dataset A filters
- `.bar-compare` – dataset B filters
- `.dataset-box` – filter group container
- `.dataset-swatch` – dataset labels (Dataset A, Dataset B)
- `.date-range-display-container` – date range and display controls wrapper
- `.radio-group` – radio button sets
- `.results-section` – output container for charts and tables
- `.loader` – spinner
- `.info-icon` – tooltip icon
- `.table-wrap` – responsive table container

Data format requirements

- `"Dividend, USD"` must be numeric
- `QUARTER` must match `/^\d{2}Q[1-4]$/`
- All other fields must match expected strings
- Missing values are handled gracefully

Customisation

Styling

Styles are now in a separate file (`css/dividend-tool.css`) loaded into the Shadow DOM. You can:

- Edit `css/dividend-tool.css` directly to customize styles
- Add CSS custom properties for theming
- Override component classes within the Shadow DOM context

Data processing customisation

- Map custom column names:

```
if (key.startsWith("dividend_amount")) key = "dividend";
if (key.includes("time_period")) key = "quarter";
```

- Custom display labels (centralized in `labelMappings` object):

```
const labelMappings = {
  sector: {
    "Media & Telecommunications": "Media & Telcos"
  },
  region: {
    "Pacific Ex China, Hong Kong & Japan": "Pacific Ex China, HK & Japan",
  },
  // Add more mappings as needed
};
```

- Custom number formats:

```
function fmt(val) {
  const opts = { minimumFractionDigits: 2, maximumFractionDigits: 2 };
  return "€" + (val / 1_000_000).toLocaleString(undefined, opts) + "m";
}
```

Adding new features

- Add new filters in selects section
- Add new chart types by modifying Chart.js config

Troubleshooting

Common issues

- **"No data-csv attribute"** → ensure `data-csv` is set
- **Data not loading** → check file path and CORS
- **Formatting issues** → confirm required columns and formats
- **Performance issues** → optimise CSV size or paginate
- **CSS not loading** → verify `css/dividend-tool.css` path is accessible

Browser console errors

- **Cannot read property of undefined** → CSV columns missing
- **Cannot use 'import.meta' outside a module** → Fixed in current version

Performance considerations

- Keep CSVs under 5 MB
- Recommended max rows: ~50,000
- Use browser caching for CSVs
- Chart.js is optimised for large datasets

Support

For enterprise integration:

- Framework wrappers (React/Vue/Angular)
- API conversion from CSV
- Performance optimisation
- Styling and theming
- Security reviews

For standard support:

- Check docs and console errors
- Validate CSV structure
- Test smaller datasets

API Reference

Constructor

```
const widget = document.createElement('dividend-analyzer');
```

Attributes

- `data-csv-url` : URL to CSV data file (required)
- `data-width` : Widget width (default: "100%")
- `data-height` : Widget height (default: "600px")
- `data-theme` : Color theme ("light" or "dark", default: "light")

Methods

- `loadData(csvUrl)` : Load new CSV data
- `updateFilters(filters)` : Apply programmatic filters
- `exportData(format)` : Export filtered data ("csv" or "json")
- `resetFilters()` : Clear all applied filters

Events

- `data-loaded` : Fired when CSV data is successfully loaded
- `filter-changed` : Fired when user changes any filter
- `comparison-updated` : Fired when comparison selection changes

CSS Custom Properties

```
dividend-analyzer {  
  --primary-color: #007bff;  
  --secondary-color: #6c757d;  
  --background-color: #ffffff;  
  --text-color: #212529;  
  --border-color: #dee2e6;  
  --hover-color: #f8f9fa;  
}
```

Customization Guide

Styling Override

```
/* Custom theme example */  
dividend-analyzer {  
  --primary-color: #28a745;  
  --secondary-color: #6f42c1;  
  --background-color: #f8f9fa;  
}  
  
/* Chart customization */  
dividend-analyzer::part(chart-container) {  
  border-radius: 8px;  
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);  
}
```

Data Format Customization

```
// Custom CSV column mapping  
const widget = document.createElement('dividend-analyzer');  
widget.setAttribute('data-column-mapping', JSON.stringify({  
  date: 'payment_date',  
  amount: 'dividend_amount',  
  company: 'company_name',  
  sector: 'industry_sector'  
})));
```

Filter Customization

```
// Programmatic filter control
widget.addEventListener('data-loaded', () => {
  widget.updateFilters({
    dateRange: ['2020-01-01', '2023-12-31'],
    companies: ['AAPL', 'MSFT'],
    sectors: ['Technology']
  });
});
```

Troubleshooting

Common Issues

Problem: Widget not loading

Solution: Check console for errors, verify CSV URL accessibility
Debug: Open browser DevTools, check Network tab for failed requests

Problem: Chart not displaying

Solution: Ensure Chart.js is loaded, check container dimensions
Debug: Verify chart canvas element exists in shadow DOM

Problem: Filters not working

Solution: Validate CSV column names match expected format
Debug: Check data parsing in browser console

Problem: Performance issues with large datasets

Solution: Implement data pagination or chunking
Debug: Monitor memory usage in DevTools Performance tab

Browser Compatibility

- Chrome 88+: Full support
- Firefox 85+: Full support
- Safari 14+: Full support
- Edge 88+: Full support
- IE: Not supported (requires Shadow DOM)

Memory Management

```
// For large datasets, consider cleanup
widget.addEventListener('beforeunload', () => {
  widget.resetFilters();
  widget.clearCache();
});
```

Performance Considerations

Dataset Size Recommendations

- **Optimal:** < 10,000 records
- **Good:** 10,000 - 50,000 records
- **Requires optimization:** > 50,000 records

Optimization Strategies

```
// Virtual scrolling for large lists
widget.setAttribute('data-virtual-scroll', 'true');

// Debounced filtering
widget.setAttribute('data-filter-debounce', '300');

// Lazy chart rendering
widget.setAttribute('data-lazy-charts', 'true');
```

Memory Usage

- Base widget: ~500KB
- Dependencies: ~2MB (Chart.js, Papa Parse)
- Data processing: ~1MB per 10,000 records

Network Optimization

```
// Gzip compression recommended
Response.headers['Content-Encoding'] = 'gzip';

// CDN usage for dependencies
widget.setAttribute('data-cdn-fallback', 'true');
```

Version history

- **v1.0:** Initial release
- **v1.1:** Added comparison mode
- **v1.2:** Responsive design
- **v1.3:** Tooltips and number formatting
- **v1.4:** Range slider and layout improvements

Licence

This project is provided as-is for educational and commercial use.
Dependencies (Papa Parse, Chart.js, noUiSlider, Popper.js) remain under their respective MIT licences.